Heaps

1. Introduction to Priority Queues using Binary Heaps

What is Priority Queue ?

A Priority Queue is a special type of queue where each element is assigned a priority, and instead of being processed in the order they arrive (like a normal queue), the element with the highest priority is always processed first.

If two elements have the same priority, they are handled based on their insertion order.

Think of it like an emergency room in a hospital patients are not treated just by arrival time, but by how critical their condition is; someone with a heart attack gets attention before someone with a cold, regardless of who came first. This makes priority queues super useful in scenarios like task scheduling, pathfinding algorithms, and real-time systems.

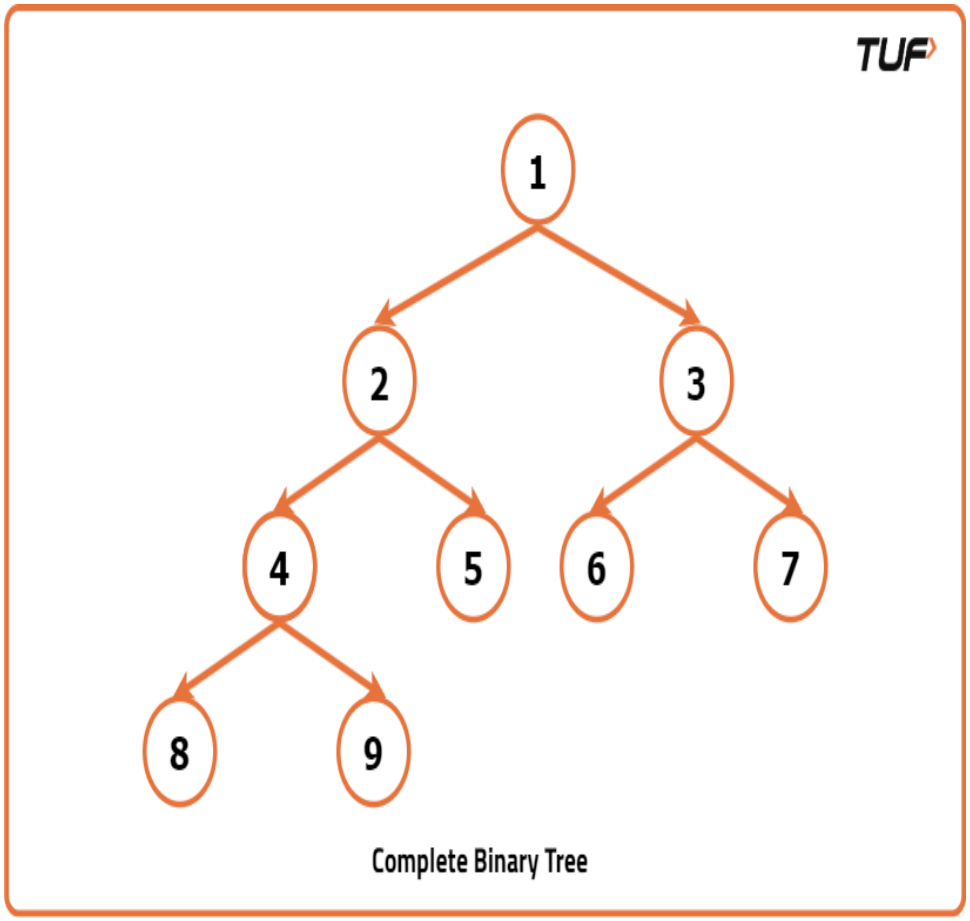
Binary Heap:

A Binary Heap is a Binary Tree that satisfies the following conditions.

- It should be a Complete Binary Tree.
- It should satisfy the Heap property.

Complete Binary Tree:

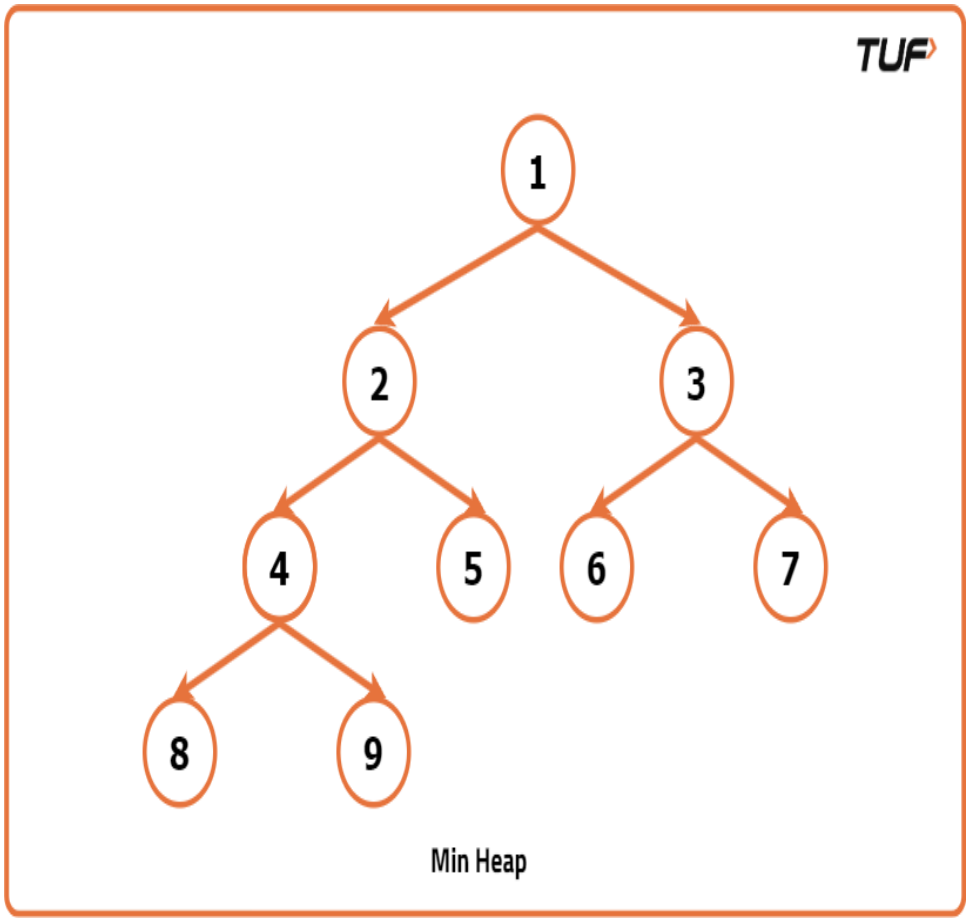
The tree in Which all the levels are completely filled except the last level and last level is filled in such a way that all the keys are as left as possible.



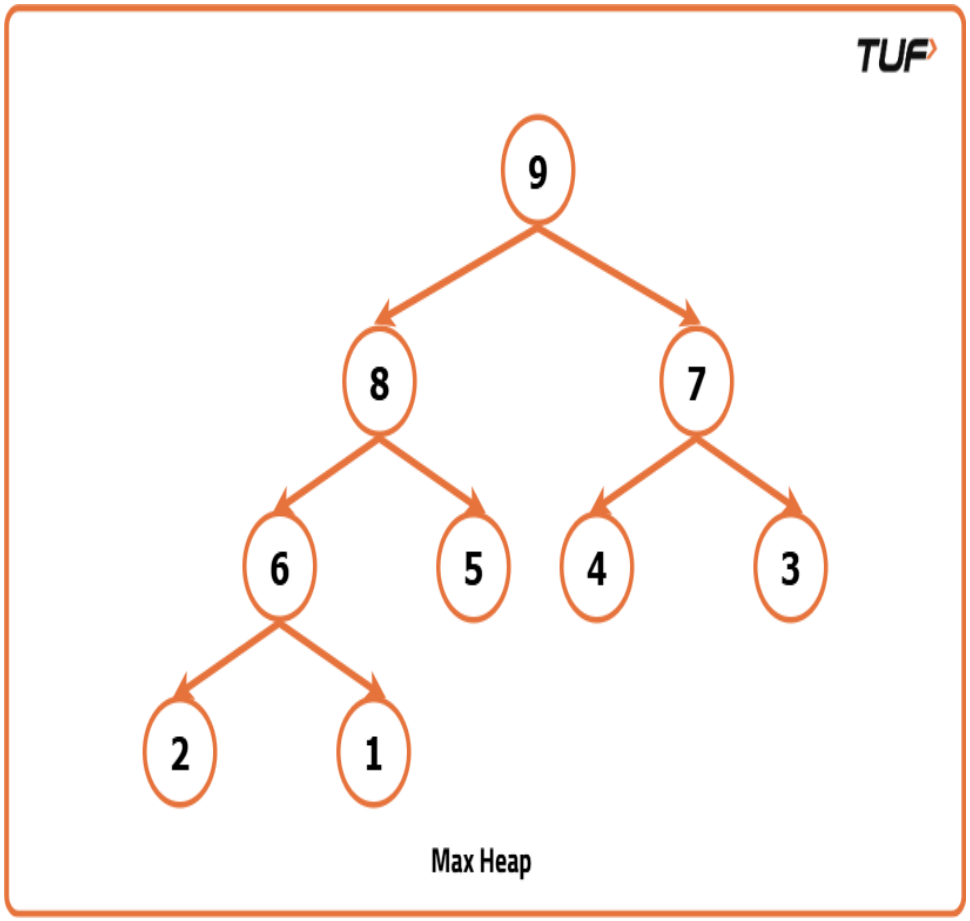
Heap Property:

Binary Heap is either a Min Heap or Max Heap. Property of the Binary Heap decides whether it is Min Heap or Max Heap.

- Min Heap property: For every node in a binary heap, if the node value is less than its right and left child's value then Binary Heap is known as Min Heap. The property of Node's value less than its children's value is known as Min Heap property. In Min Heap, the root value is always the Minimum value in Heap.



- Max Heap property: For every node in a binary heap, if the node value is greater than its right and left child's value then Binary Heap is known as Max Heap. The property of being Node's value greater than its children's value is known as Max Heap property. In Max Heap, the root value is always the maximum value in Heap.



Representation of the Binary Heap

A Binary Heap is represented as an array.

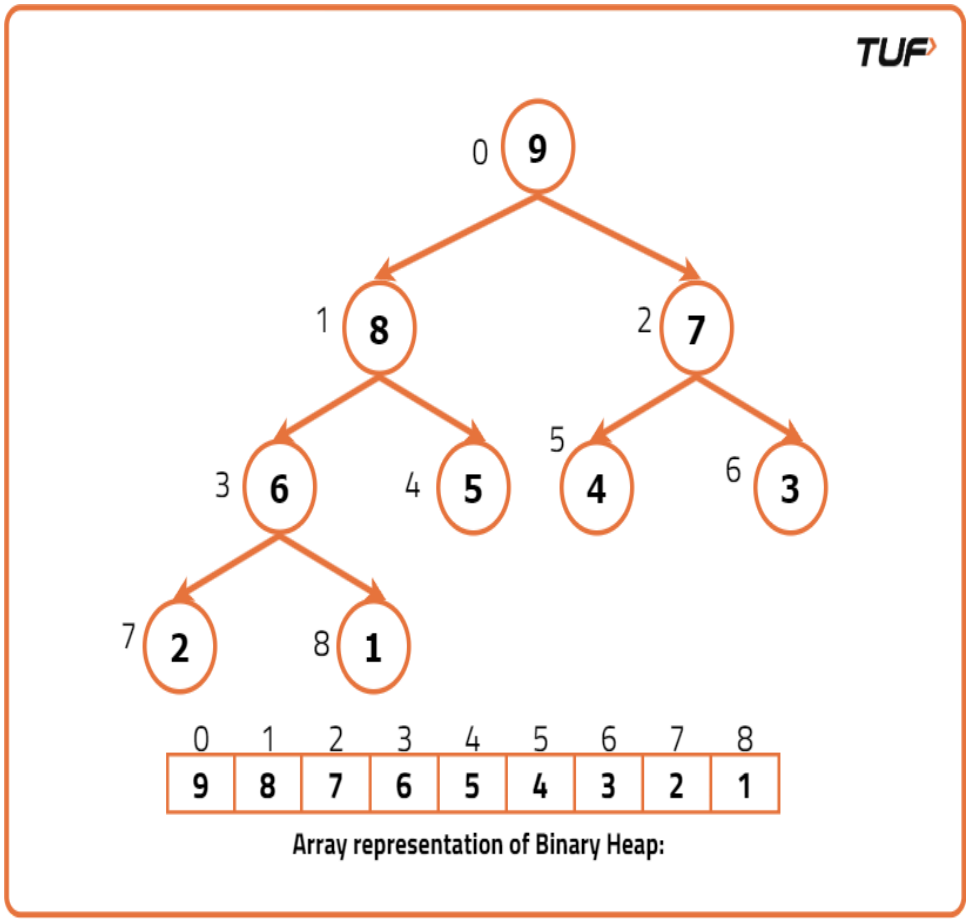
- The root value is at arr[0]

The below table summarizes how the node and its children are stored in an array.

Node index	Left child Index	Right Child Index
i	2*i+1	2*i+2

If the child is at index **i**, then its parent index can be found using the below formula.

Child's Index	Parent's Index
i	(i-1)/2



Operations Associated with Min Heap

- Insert()
- Heapify()
- getMin()

- ExtractMin()
- Delete()
- Decreasekey()

Let's see how these operations are done Using Min Heap.

Note: The Binary heap should always be a complete binary tree and should satisfy the corresponding heap property (Min / Max). If any of the two conditions are disturbed we should make necessary modifications in a heap to satisfy the two conditions.

Insert():

- Insert operation inserts a new key in the Binary Heap.

Steps Followed for inserting the key in Binary Heap:

- First Insert the key at the first vacant position from the left on the last level of the heap. IF the last level is completely filled, then insert the key as the left-most element in the next level.
- Inserting a new key at the first vacant position in the last level preserves the Complete binary tree property, The Min heap property may get affected we need to check for it.
- If the inserted key parent is less than the key, then the Min heap property is also not violated.
- If the parent is greater than the inserted key value, then swap the values. Now the heap property may get violated at the parent node. So repeat the same process until the heap property is satisfied.
- Inserting an element takes O(logN) Time Complexity. Below shows the animation of how -1 is inserted into a Binary heap by following above described steps.

Heapify():

- Suppose there exists a Node at some index i, where the Minheap property is Violated.
- We assume that all the subtrees of the tree rooted at index i are valid binary heaps.
- The Heapify function is used to restore the Minheap, by fixing the violation.

Steps to be followed for Heapify:

- First find out the Minimum among the Violated Node, left, and right child of Violated Node.

- If the Minimum among them is the left child, then swap the Violated Node value with the Left child value and recursively call the function on the left Child.
- If the Minimum among them is the right child, then swap the Violated Node value with the right child value and recursively call the function right Child.
- Recursive call stops when the heap property is not violated.

getMin():

- It returns the minimum value in the Binary Heap.
- As we all know, the root Node is the Minimum value in Min Heap. Simply return the arr[0].

ExtractMin():

- This removes the Minimum element from the heap.

Steps to be followed to Remove Minimum value/root Node:

- Copy the last Node value to the Root Node, and decrease the size, this means that the root value is now deleted from the heap, and the size is decreased by 1.
- By doing the above step we ensure that the Complete binary tree property is not violated, as we are copying the last node value to the root node value, the Min Heap property gets violated.
- Call the Heapify function to convert it into a valid heap.

Decreasekey():

- Given an index and a value, we need to update the value at the index with the given value. We assume that the given value is less than the existing value at that index.

Steps to be followed for Decreasekey():

- Let's the index be i and the value be new_val. Update existing value at index i with new_val i.e arr[i] = new_val.
- By performing the above step, the Complete binary tree property is not violated, but the Min heap property may get violated.
- As the new_val we are inserting is less than the previously existing value, the min-heap property is not violated in subtrees of this rooted tree.
- It may get violated in its ancestors, so as we do in insert operation, check the value of a current node with its parent node, if it violates the min-heap property

Swap the nodes and recursively do the same.

Delete() :

- Given an index, delete the value at that index from the min-heap.

Steps to be followed for Delete operation():

- First, update the value at the index that needs to be deleted with INT_MIN.
- Now call the Decreasekey() function at the index which is need to be deleted. As the value at the index is the least, it reaches the top.
- Now call the ExtractMin() operation which deletes the root node in Minheap.
- In this way, the desired index value is deleted from the Minheap.

Code:

C++JavaPythonJavaScript

```
#include <bits/stdc++.h>

#include
using namespace std;

class BinaryHeap {
public:
    // Maximum elements that can be stored in heap
    int capacity;

    // Current no of elements in heap
    int size;

    // Array for storing the keys
    int* arr;

    BinaryHeap(int cap) {
        // Assigning the capacity
        capacity = cap;

        // Initially size of heap is zero
        size = 0;

        // Creating an array
        arr = new int[capacity];
```

```
}

// Returns the parent of ith Node
int parent(int i) {
    return (i - 1) / 2;
}

// Returns the left child of ith Node
int left(int i) {
    return 2 * i + 1;
}

// Returns the right child of the ith Node
int right(int i) {
    return 2 * i + 2;
}

// Insert a new key x
void Insert(int x) {
    if (size == capacity) {
        cout << "Binary Heap Overflow" << endl;
        return;
    }

    // Insert new element at end
    arr[size] = x;

    // Store the index, for checking heap property
    int k = size;

    // Increase the size
    size++;

    // Fix the min heap property
    while (k != 0 && arr[parent(k)] > arr[k]) {
        swap(&arr[parent(k)], &arr[k]);
        k = parent(k);
    }
}

void Heapify(int ind) {
```

```
        // Right child
        int ri = right(ind);

        // Left child
        int li = left(ind);

        // Initially assume violated value is minimum
        int smallest = ind;

        if (li < size && arr[li] < arr[smallest])
            smallest = li;

        if (ri < size && arr[ri] < arr[smallest])
            smallest = ri;

        // If the Minimum among the three nodes is not the
parent itself,
        // then swap and call Heapify recursively
        if (smallest != ind) {
            swap(&arr[ind], &arr[smallest]);
            Heapify(smallest);
        }
    }

    int getMin() {
        return arr[0];
    }

    int ExtractMin() {
        if (size <= 0)
            return INT_MAX;

        if (size == 1) {
            size--;
            return arr[0];
        }

        int mini = arr[0];

        // Copy last Node value to root Node
        arr[0] = arr[size - 1];
```

```
        size--;

        // Call heapify on root node
        Heapify(0);

        return mini;
    }

    void Decreasekey(int i, int val) {
        // Updating the new value
        arr[i] = val;

        // Fixing the Min heap
        while (i != 0 && arr[parent(i)] > arr[i]) {
            swap(&arr[parent(i)], &arr[i]);
            i = parent(i);
        }
    }

    void Delete(int i) {
        Decreasekey(i, INT_MIN);
        ExtractMin();
    }

    void swap(int* x, int* y) {
        int temp = *x;
        *x = *y;
        *y = temp;
    }

    void print() {
        for (int i = 0; i < size; i++)
            cout << arr[i] << " ";
        cout << endl;
    }
};

int main() {
    BinaryHeap h(20);
    h.Insert(4);
```

```
    h.Insert(1);
    h.Insert(2);
    h.Insert(6);
    h.Insert(7);
    h.Insert(3);
    h.Insert(8);
    h.Insert(5);

    cout << "Min value is " << h.getMin() << endl;

    h.Insert(-1);
    cout << "Min value is " << h.getMin() << endl;

    h.Decreasekey(3, -2);
    cout << "Min value is " << h.getMin() << endl;

    h.ExtractMin();
    cout << "Min value is " << h.getMin() << endl;

    h.Delete(0);
    cout << "Min value is " << h.getMin() << endl;

    return 0;
}


}
```

Time Complexities :

Function	Time Complexity
Insert() :	O(logN)
Heapify()	O(logN)

getMin()	O(1)
ExtractMin()	O(logN)
Decreasekey()	O(logN)
Delete()	O(logN)

- **Data members ka role**

```
capacity → max elements
size     → current elements
arr[]    → heap array
```

- **INSERT(x) — sabse pehle samjho**

Steps:

1. New element **end me daalo**
2. Phir **upar bubble up** karao jab tak heap property na ban jaaye

```
arr[size] = x;
int k = size;
size++;
```

Bubble-up logic:

```
while (k != 0 && arr[parent(k)] > arr[k]) {
    swap(parent, k);
```

```
    k = parent(k);
}
```

👉 Jab child chhota ho parent se → swap

🧪 **Example**

Insert: `-1`

Before:

`[1, 3, 2, 6, 7, 4, 8, 5]`

After inserting at end:

`[1,3,2,6,7,4,8,5,-1]`

Bubble up:

`-1` ↔ `6`
`-1` ↔ `3`
`-1` ↔ `1`

Final:

`[-1,1,2,3,7,4,8,5,6]`

- **HEAPIFY(ind) — neeche fix karna**

  Jab root ya koi middle element galat ho jaaye

Steps:

1. Parent, left, right me **minimum** nikaalo

2. Agar parent minimum nahi → swap
3. Recursively neeche jao

```
smallest = ind;
if (left < size && arr[left] < arr[smallest]) smallest = left;
if (right < size && arr[right] < arr[smallest]) smallest = right;

if (smallest != ind) {
    swap(ind, smallest);
    Heapify(smallest);
}
```

🔹 **ExtractMin() — root hataana**

Steps:

1. Root store karo (answer)
2. Last element ko root pe le aao
3. Size--
4. Heapify(0)

```
mini = arr[0];
arr[0] = arr[size-1];
size--;
Heapify(0);
```

👉 Root hataane ke baad heap property toot jaati hai → Heapify se fix

---

🔹 **getMin()**

```
return arr[0];
```

Simple: root hamesha minimum hota hai

- **DecreaseKey(i, val)**

  Jab kisi index ki value **kam** karni ho

Steps:

1. Value update
2. Bubble-up (same as insert logic)

```
arr[i] = val;
while (i != 0 && arr[parent(i)] > arr[i]) {
    swap(i, parent(i));
    i = parent(i);
}
```

- **Delete(i) — clever trick**

Heap me direct delete mushkil hota hai
Isliye trick use ki:

```
Decreasekey(i, INT_MIN);
ExtractMin();
```

👉 Node ko sabse chhota bana diya
👉 Root tak aa gaya
👉 ExtractMin se nikal gaya

- **swap function**

Tumne pointer-based swap likha hai:

```
void swap(int* x, int* y)
```

Ye bhi sahi hai, STL `swap()` bhi use kar sakte ho.

🔹 **Main function ka flow**

```
Insert → heap banti hai
getMin → root
DecreaseKey → bubble up
ExtractMin → root remove
Delete → decrease + extract
```

🔑 **Sabse important intuition (yaad rakhna)**

- **Insert / DecreaseKey → bubble UP**
- **ExtractMin / Delete → bubble DOWN (Heapify)**

2. Min Heap and Max Heap Implementation

A heap is a complete binary tree that follows a special order property.
In a **Min Heap**, the smallest element is always at the root, meaning every parent node is smaller than its children.
In a **Max Heap**, the largest element is always at the root, meaning every parent node is larger than its children.
Heaps are commonly used to efficiently get the minimum or maximum element, such as in priority queues.

Example:
If we insert elements `10, 5, 20`

- Min Heap root will be 5
- Max Heap root will be 20

Approach 1: Min Heap Implementation

Algorithm

1. Use an array to represent the heap.
2. For insertion:
    - Insert the element at the end.
    - Move it up (heapify up) while parent is greater than current.
3. For deletion of minimum:
    - Replace root with last element.
    - Remove last element.
    - Move root down (heapify down) to maintain heap property.
4. Root always stores the minimum element.

Code

```
#include <bits/stdc++.h>
using namespace std;

class MinHeap{
    vector<int> heap;

    void heapifyUp(int index){
        while(index>0){
            int parent=(index-1)/2;
            if(heap[parent]>heap[index]){
                swap(heap[parent],heap[index]);
```

```
                index=parent;
            }else break;
        }
    }

    void heapifyDown(int index){
        int n=heap.size();
        while(true){
            int left=2*index+1;
            int right=2*index+2;
            int smallest=index;
            if(left<n && heap[left]<heap[smallest]) smallest=left;
            if(right<n && heap[right]<heap[smallest]) smallest=right;
            if(smallest!=index){
                swap(heap[index],heap[smallest]);
                index=smallest;
            }else break;
        }
    }

public:
    void insert(int val){
        heap.push_back(val);
        heapifyUp(heap.size()-1);
    }

    void removeMin(){
        if(heap.empty()) return;
        heap[0]=heap.back();
        heap.pop_back();
        heapifyDown(0);
    }

    int getMin(){
        if(heap.empty()) return -1;
        return heap[0];
    }
};
```

Complexity Analysis

- Insertion: `O(log n)`
- Deletion: `O(log n)`
- Get Minimum: `O(1)`
- Space Complexity: `O(n)`

Approach 2: Max Heap Implementation

Algorithm

1. Use an array to represent the heap.
2. For insertion:
    - Insert the element at the end.
    - Move it up (heapify up) while parent is smaller than current.
3. For deletion of maximum:
    - Replace root with last element.
    - Remove last element.
    - Move root down (heapify down) to maintain heap property.
4. Root always stores the maximum element.

Code

```
#include <bits/stdc++.h>
using namespace std;

class MaxHeap{
    vector<int> heap;
```

```
    void heapifyUp(int index){
        while(index>0){
            int parent=(index-1)/2;
            if(heap[parent]<heap[index]){
                swap(heap[parent],heap[index]);
                index=parent;
            }else break;
        }
    }

    void heapifyDown(int index){
        int n=heap.size();
        while(true){
            int left=2*index+1;
            int right=2*index+2;
            int largest=index;
            if(left<n && heap[left]>heap[largest]) largest=left;
            if(right<n && heap[right]>heap[largest]) largest=right;
            if(largest!=index){
                swap(heap[index],heap[largest]);
                index=largest;
            }else break;
        }
    }

public:
    void insert(int val){
        heap.push_back(val);
        heapifyUp(heap.size()-1);
    }

    void removeMax(){
        if(heap.empty()) return;
        heap[0]=heap.back();
        heap.pop_back();
        heapifyDown(0);
    }
```

```
    int getMax(){
        if(heap.empty()) return -1;
        return heap[0];
    }
};
```

Complexity Analysis

- Insertion: `O(log n)`
- Deletion: `O(log n)`
- Get Maximum: `O(1)`
- Space Complexity: `O(n)`

3. Check if an Array Represents a Min Heap

Given an array of integers `nums`, check whether it represents a binary min-heap.
In a binary min-heap, every parent node has a value less than or equal to its children. The tree is complete, and this property must hold for all nodes.

Example:
Input: `nums = [10, 20, 30, 21, 23]`
Here, every parent value is smaller than its children, so it is a valid min-heap.

Input: `nums = [10, 20, 30, 25, 15]`
The node `20` has a child `15` which is smaller, so the min-heap property is violated.

Approach 1

Algorithm

We store a binary heap in an array.
For an index `i`:

- Left child is at `2*i + 1`
- Right child is at `2*i + 2`

Leaf nodes do not have children, so they cannot violate the heap property.
In an array of size n, all non-leaf nodes are from index `0` to `(n/2) - 1`.

Steps:

1. Iterate from index `0` to `(n/2) - 1`.
2. For each index:
    - Calculate left child index.
    - If left child exists and parent value is greater, return false.
    - Calculate right child index.
    - If right child exists and parent value is greater, return false.
3. If no violation is found, return true.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    bool isMinHeap(vector<int>& nums) {
        int n = nums.size();

        for (int i = 0; i <= (n / 2) - 1; i++) {
            int left = 2 * i + 1;
```

```
            if (left < n && nums[i] > nums[left]) {
                return false;
            }

            int right = 2 * i + 2;
            if (right < n && nums[i] > nums[right]) {
                return false;
            }
        }
        return true;
    }
};

int main() {
    Solution obj;
    vector<int> nums = {10, 20, 30, 21, 23};
    cout << (obj.isMinHeap(nums) ? "true" : "false") << endl;
    return 0;
}
```

Complexity Analysis

- Time Complexity: $O(n)$ because we check at most $n/2$ nodes.
- Space Complexity: $O(1)$ since no extra space is used.

4. Convert Min Heap to Max Heap

Given an array that represents a binary **Min Heap**, convert it into a **Max Heap**.
In a Min Heap, every parent node is smaller than its children, while in a Max Heap, every parent node is greater than its children.
The array already represents a complete binary tree, so we only need to rearrange elements to satisfy the Max Heap property.

Example:
Input: `arr = {3, 5, 9, 6, 8, 20, 10, 12, 18, 9}`
After conversion, the root becomes the maximum element, and all parent nodes are greater than their children.

Algorithm

We do not care that the input is a Min Heap. We directly build a Max Heap from the array.

Key points:

- In an array of size n, all non-leaf nodes are from index `0` to `(n-2)/2`.
- Leaf nodes already satisfy heap property.
- We start heapifying from the **last non-leaf node** and move upward to the root.

Steps:

1. Find the last non-leaf node at index `(n-2)/2`.
2. For each index from `(n-2)/2` down to `0`:
    - Apply **MaxHeapify** on that index.
3. MaxHeapify compares a node with its children.
    - If any child is larger, swap with the largest child.
    - Recursively heapify the affected subtree.
4. After finishing, the array becomes a valid Max Heap.

Code

```
#include <bits/stdc++.h>
using namespace std;

void MaxHeapify(int arr[], int i, int N){
    int left = 2*i + 1;
    int right = 2*i + 2;
```

```
    int largest = i;

    if(left < N && arr[left] > arr[largest])
        largest = left;
    if(right < N && arr[right] > arr[largest])
        largest = right;

    if(largest != i){
        swap(arr[i], arr[largest]);
        MaxHeapify(arr, largest, N);
    }
}

void convertMaxHeap(int arr[], int N){
    for(int i = (N - 2) / 2; i >= 0; i--){
        MaxHeapify(arr, i, N);
    }
}

void printArray(int arr[], int n){
    for(int i = 0; i < n; i++)
        cout << arr[i] << " ";
}

int main(){
    int arr[] = {3, 5, 9, 6, 8, 20, 10, 12, 18, 9};
    int N = sizeof(arr) / sizeof(arr[0]);

    printArray(arr, N);
    cout << endl;

    convertMaxHeap(arr, N);

    printArray(arr, N);
    return 0;
}
```

- Time Complexity: $O(n)$ because heapifying all internal nodes takes linear time.
- Space Complexity: $O(n)$ due to recursion stack in worst case.

5. Kth Largest / Smallest Element in an Array

Given an array `nums` and an integer `k`, return the **kth largest element** in the array.
The kth largest element means the element which will appear at position `k` if the array is sorted in descending order.

Example:
Input: `nums = [1, 2, 3, 4, 5], k = 2`
Sorted (descending): `[5, 4, 3, 2, 1]`
The 2nd largest element is `4`.

Input: `nums = [-5, 4, 1, 2, -3], k = 5`
Sorted (descending): `[4, 2, 1, -3, -5]`
The 5th largest element is `-5`.

Approach 1: Brute Force (Using Min Heap)

Algorithm

We maintain a **min-heap of size K** that stores the K largest elements seen so far.

Steps:

1. Create a min-heap.
2. Insert the first k elements of the array into the heap.
3. For each remaining element:
    - If the element is greater than the top of the heap (smallest among K largest),
        - Remove the top element.
        - Insert the current element.
4. After processing all elements, the top of the heap is the kth largest element.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int kthLargestElement(vector<int>& nums, int k) {
        priority_queue<int, vector<int>, greater<int>> pq;

        for(int i = 0; i < k; i++)
            pq.push(nums[i]);

        for(int i = k; i < nums.size(); i++) {
            if(nums[i] > pq.top()) {
                pq.pop();
                pq.push(nums[i]);
            }
        }
        return pq.top();
    }
};

int main() {
    vector<int> nums = {-5, 4, 1, 2, -3};
    int k = 5;

    Solution sol;
    cout << sol.kthLargestElement(nums, k);
    return 0;
}
```

Complexity Analysis

- Time Complexity: `O(N log K)`
- Space Complexity: `O(K)`

Approach 2: Optimal (Quick Select)

Algorithm

This approach is based on partitioning, similar to Quick Sort.

Steps:

1. Choose a random pivot index in the current range.
2. Partition the array such that:
    - Elements greater than pivot are on the left.
    - Elements smaller or equal are on the right.
3. Place the pivot in its correct position.
4. If pivot index equals `k-1`, return the pivot value.
5. If pivot index is greater than `k-1`, repeat on the left part.
6. If pivot index is smaller than `k-1`, repeat on the right part.
7. Continue until the kth largest element is found.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int kthLargestElement(vector<int>& nums, int k) {
        if(k > nums.size()) return -1;

        int left = 0, right = nums.size() - 1;

        while(true) {
            int pivotIndex = randomIndex(left, right);
            pivotIndex = partition(nums, pivotIndex, left, right);
```

```
            if(pivotIndex == k - 1)
                return nums[pivotIndex];
            else if(pivotIndex > k - 1)
                right = pivotIndex - 1;
            else
                left = pivotIndex + 1;
        }
    }

private:
    int randomIndex(int left, int right) {
        return left + rand() % (right - left + 1);
    }

    int partition(vector<int>& nums, int pivotIndex, int left, int
right) {
        int pivot = nums[pivotIndex];
        swap(nums[left], nums[pivotIndex]);

        int ind = left + 1;
        for(int i = left + 1; i <= right; i++) {
            if(nums[i] > pivot) {
                swap(nums[i], nums[ind]);
                ind++;
            }
        }
        swap(nums[left], nums[ind - 1]);
        return ind - 1;
    }
};

int main() {
    vector<int> nums = {-5, 4, 1, 2, -3};
    int k = 5;

    Solution sol;
    cout << sol.kthLargestElement(nums, k);
    return 0;
```

}

Complexity Analysis

- Time Complexity:
    - Average Case: O(N)
      In the average case (when the pivot is chosen randomly):
    - Assuming the array gets divided into two equal parts, with every partitioning step, the search range is reduced by half. Thus, the time complexity is O(N + N/2 + N/4 + ... + 1) = O(N).
    -
    - In the worst-case scenario (when the element at the left or right index is chosen as the pivot):
    - In such cases, the array is divided into two unequal halves, and the search range is reduced by one element with every partitioning step. Thus, the time complexity is O(N + N-1 + N-2 + ... + 1) = O(N_2). However, the probability of this worst-case scenario is negligible.
    - Worst Case: O(N^2)
- Space Complexity: O(1)

6. Sort K Sorted Array

Given an array `arr[]` and an integer k, the array is such that every element is at most k positions away from its correct position in the fully sorted array.
Your task is to completely sort the array.

Example:
Input: `arr = [6, 5, 3, 2, 8, 10, 9], k = 3`
After sorting, the array becomes `[2, 3, 5, 6, 8, 9, 10]`.

Input: `arr = [1, 4, 5, 2, 3, 6, 7, 8, 9, 10], k = 2`
After sorting, the array becomes `[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]`.

Approach 1: Brute Force Approach

Algorithm

In this approach, we ignore the fact that the array is nearly sorted.
We treat it like a normal unsorted array and sort the entire array.

Steps:

1. Take the entire array as input.
2. Apply a standard sorting algorithm.
3. Return the fully sorted array.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    vector<int> sortNearlySortedArray(vector<int>& arr, int k) {
        sort(arr.begin(), arr.end());
        return arr;
    }
};

int main() {
    vector<int> arr = {6, 5, 3, 2, 8, 10, 9};
    int k = 3;

    Solution obj;
    vector<int> result = obj.sortNearlySortedArray(arr, k);

    for(int x : result)
        cout << x << " ";
    return 0;
}
```

- Time Complexity: `O(n log n)` because the entire array is sorted.

- Space Complexity: $O(1)$ for in-place sort (ignoring recursion stack).

Approach 2: Optimal Approach (Using Min Heap)

Algorithm

Since the array is k-sorted, the smallest element must be present within the first $k + 1$ elements.

Steps:

1. Create a min heap.
2. Insert the first $k + 1$ elements into the heap.
3. For each remaining element:
    - Remove the minimum element from the heap and add it to the result.
    - Insert the current element into the heap.
4. After processing all elements, remove all remaining elements from the heap and add them to the result.
5. Return the result array.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    vector<int> sortNearlySortedArray(vector<int>& arr, int k) {
        priority_queue<int, vector<int>, greater<int>> minHeap;
        vector<int> result;

        for(int i = 0; i <= k && i < arr.size(); i++)
            minHeap.push(arr[i]);
```

```
        for(int i = k + 1; i < arr.size(); i++) {
            result.push_back(minHeap.top());
            minHeap.pop();
            minHeap.push(arr[i]);
        }

        while(!minHeap.empty()) {
            result.push_back(minHeap.top());
            minHeap.pop();
        }
        return result;
    }
};

int main() {
    vector<int> arr = {6, 5, 3, 2, 8, 10, 9};
    int k = 3;

    Solution obj;
    vector<int> sortedArr = obj.sortNearlySortedArray(arr, k);

    for(int x : sortedArr)
        cout << x << " ";
    return 0;
}
```

Complexity Analysis

- Time Complexity: `O(n log k)` because each heap operation takes `O(log k)`.
- Space Complexity: `O(k)` due to the min heap of size `k + 1`.

7. Merge M Sorted Lists

Given an array `heads` where each element is the head of a sorted linked list, merge all these sorted linked lists into one single sorted linked list and return its head.
All input linked lists are already sorted in ascending order.

Example:
Input: `heads = [[1 -> 2 -> 3 -> 4], [-4 -> -3], [-5 -> -3 -> 1 -> 2 -> 3 -> 4]]`
After merging all nodes in sorted order, the final list becomes:
`-5 -> -4 -> -3 -> -3 -> 1 -> 1 -> 2 -> 2 -> 3 -> 3 -> 4 -> 4`

Input: `heads = [[-5 -> -4 -> -1], [10 -> 11 -> 12]]`
Output: `-5 -> -4 -> -1 -> 10 -> 11 -> 12`

Approach 1: Brute Force Approach

Algorithm

In this approach, we do not take advantage of the fact that the lists are already sorted.

Steps:

1. Create an empty array to store all node values.
2. Traverse each linked list one by one.
3. Push every node's value into the array.
4. Sort the array.
5. Create a new linked list using the sorted values.
6. Return the head of the newly created list.

Code

```
#include <bits/stdc++.h>

using namespace std;
```

```
struct ListNode {
    int val;
    ListNode *next;
    ListNode(int x) : val(x), next(NULL) {}
};

class Solution {
public:
    ListNode* mergeKSortedLists(vector<ListNode*>& lists) {
        vector<int> values;

        for(auto list : lists) {
            while(list != NULL) {
                values.push_back(list->val);
                list = list->next;
            }
        }

        sort(values.begin(), values.end());

        ListNode* dummy = new ListNode(0);
        ListNode* curr = dummy;
```

```
        for(int v : values) {
            curr->next = new ListNode(v);
            curr = curr->next;
        }
        return dummy->next;
    }
};

int main() {
    ListNode* a = new ListNode(1);
    a->next = new ListNode(4);
    a->next->next = new ListNode(5);

    ListNode* b = new ListNode(1);
    b->next = new ListNode(3);
    b->next->next = new ListNode(4);

    ListNode* c = new ListNode(2);
    c->next = new ListNode(6);

    vector<ListNode*> lists = {a, b, c};
```

```
    Solution obj;
    ListNode* res = obj.mergeKSortedLists(lists);

    while(res) {
        cout << res->val << " ";
        res = res->next;
    }
    return 0;
}
```

Complexity Analysis

- Time Complexity: `O(N log N)` where `N` is the total number of nodes.
- Space Complexity: `O(N)` for storing all node values.

Approach 2: Optimal Approach (Using Min Heap)

Algorithm

This approach uses the fact that all lists are already sorted.

Steps:

1. Create a min-heap that stores pointers to nodes.
2. Push the head of each non-empty linked list into the heap.
3. Create a dummy node to build the result list.
4. While the heap is not empty:

- Extract the node with the smallest value.
- Attach it to the result list.
- If the extracted node has a next node, push that next node into the heap.

5. Return the node next to the dummy head.

Code

```
#include <bits/stdc++.h>
using namespace std;


struct ListNode {
    int val;
    ListNode* next;
    ListNode(int x) : val(x), next(NULL) {}
};


class Compare {
public:
    bool operator()(ListNode* a, ListNode* b) {
        return a->val > b->val;
    }
// kya A ko B ke baad aana chahiye?
};
```

```
class Solution {
public:
    ListNode* mergeKLists(vector<ListNode*>& lists) {
        priority_queue<ListNode*, vector<ListNode*>, Compare> pq;

        for(auto list : lists) {
            if(list != NULL)
                pq.push(list);
        }

        ListNode* dummy = new ListNode(0);
        ListNode* tail = dummy;

        while(!pq.empty()) {
            ListNode* node = pq.top();
            pq.pop();

            tail->next = node;
            tail = tail->next;

            if(node->next != NULL)
                pq.push(node->next);
```

```
        }
        return dummy->next;
    }
};

int main() {
    ListNode* l1 = new ListNode(1);
    l1->next = new ListNode(4);
    l1->next->next = new ListNode(5);

    ListNode* l2 = new ListNode(1);
    l2->next = new ListNode(3);
    l2->next->next = new ListNode(4);

    ListNode* l3 = new ListNode(2);
    l3->next = new ListNode(6);

    vector<ListNode*> lists = {l1, l2, l3};

    Solution sol;
    ListNode* res = sol.mergeKLists(lists);

    while(res) {
```

```
        cout << res->val << " ";
        res = res->next;
    }
    return 0;
}
```

Complexity Analysis

- Time Complexity: `O(N log K)` where `N` is total nodes and `K` is number of lists.
- Space Complexity: `O(K)` for the min heap.

8. Replace Elements by Its Rank in the Array

Given an array of integers, replace each element with its **rank** when the array is sorted.
The smallest element gets rank 1.
If there are duplicate values, they must get the **same rank**.

Example:
Input: `20 15 26 2 98 6`
Sorted array: `2 6 15 20 26 98`
So ranks are: `2->1, 6->2, 15->3, 20->4, 26->5, 98->6`
Output: `4 3 5 1 6 2`

Input: `1 5 8 15 8 25 9`
Sorted array: `1 5 8 8 9 15 25`
Duplicate 8 gets the same rank.
Output: `1 2 3 5 3 6 4`

Approach 1: Brute Force Approach

Algorithm

For each element, we count how many **unique elements are smaller than it**.

Steps:

1. Initialize an empty result array.
2. For each element in the array:
    - Create an empty set.
    - Traverse the array again.
    - If another element is smaller, insert it into the set.
3. Rank = size of the set + 1.
4. Store the rank in the result array.
5. Return the result.

Code

```
#include <bits/stdc++.h>

using namespace std;


class Solution {

public:

    vector<int> replaceWithRank(vector<int>& arr) {

        vector<int> rankArr;
```

```
        for(int i = 0; i < arr.size(); i++) {
            unordered_set<int> smaller;

            for(int j = 0; j < arr.size(); j++) {
                if(arr[j] < arr[i]) {
                    smaller.insert(arr[j]);
                }
            }
            rankArr.push_back(smaller.size() + 1);
        }
        return rankArr;
    }
};

int main() {
    vector<int> arr = {20, 15, 26, 2, 98, 6};
    Solution sol;
    vector<int> res = sol.replaceWithRank(arr);

    for(int x : res)
        cout << x << " ";
    return 0;
}
```

Complexity Analysis

- Time Complexity: $O(N^2)$ because we compare every element with every other element.
- Space Complexity: $O(N)$ due to the set and result array.

Approach 2: Optimal Approach

Algorithm

We sort the array and assign ranks in increasing order, skipping duplicates.

Steps:

1. Create a copy of the original array.
2. Sort the copied array.
3. Use a map to store the rank of each unique element.
4. Traverse the sorted array and assign ranks starting from 1.
5. Traverse the original array and replace each element using the rank map.
6. Return the result array.

Code

```
#include <bits/stdc++.h>

using namespace std;


class Solution {

public:

    vector<int> replaceWithRank(vector<int>& arr) {
```

```
        vector<int> sortedArr = arr;
        sort(sortedArr.begin(), sortedArr.end());

        unordered_map<int,int> rankMap;
        int rank = 1;

        for(int num : sortedArr) {
            if(rankMap.find(num) == rankMap.end()) {
                rankMap[num] = rank;
                rank++;
            }
        }

        vector<int> result;
        for(int num : arr) {
            result.push_back(rankMap[num]);
        }
        return result;
    }
};

int main() {
    vector<int> arr = {1, 5, 8, 15, 8, 25, 9};
```

```
    Solution sol;
    vector<int> res = sol.replaceWithRank(arr);


    for(int x : res)
        cout << x << " ";
    return 0;
}
```

Complexity Analysis

- Time Complexity: `O(N log N)` due to sorting.
- Space Complexity: `O(N)` for the map and copied array.

Approach 3: Heap Based Approach (Using Max Heap)

Algorithm

We assign ranks by always picking the **maximum element first**.

Steps:

1. Create a max heap storing `(value, index)` pairs.
2. Push all array elements with their indices into the heap.
3. Initialize `rank = n`.
4. While heap is not empty:
    - Pop the maximum element.
    - Assign current rank to its original index.

- Decrease rank.

5. The array gets transformed into ranks directly.

Code

```
#include <bits/stdc++.h>

using namespace std;


void transform(vector<int>& input) {

    priority_queue<pair<int,int>> maxHeap;


    for(int i = 0; i < input.size(); i++) {

        maxHeap.push({input[i], i});

    }


    int rank = input.size();


    while(!maxHeap.empty()) {

        input[maxHeap.top().second] = rank;

        maxHeap.pop();

        rank--;

    }

}
```

```
int main() {

    vector<int> input = {10, 8, 15, 12, 6, 20, 1};


    transform(input);


    for(int x : input)

        cout << x << " ";

    return 0;

}
```

Complexity Analysis

- Time Complexity: `O(N log N)`
- Space Complexity: `O(N)`

9. Task Scheduler

You are given a list of tasks represented by uppercase English letters (`A` to `Z`) and an integer n which represents the cooldown period between two same tasks.
Each task takes exactly 1 unit of CPU time.
The same task must be separated by at least n intervals, during which the CPU can either execute another task or stay idle.
Return the **minimum number of CPU intervals** required to finish all tasks.

Example:
Input: `tasks = ["A","A","A","B","B","B"], n = 2`

One valid order: `A -> B -> idle -> A -> B -> idle -> A -> B`
Total intervals = 8

Input: `tasks = ["A","C","A","B","D","B"], n = 1`
One valid order: `A -> B -> C -> D -> A -> B`
Total intervals = 6

Approach: Simulation Using Max Heap

Algorithm

We simulate task execution while respecting the cooldown constraint.

Steps:

1. Count the frequency of each task.
2. Push all task frequencies into a max heap.
3. While the heap is not empty:
    - Create a temporary list for tasks used in the current cycle.
    - Run a cycle of length `n + 1`.
    - In each slot:
        - If heap is not empty, pop the task with highest frequency.
        - Execute it, decrease its count, and store it if it still remains.
        - Increase total time.
    - Push remaining tasks from the temporary list back into the heap.
    - If heap is not empty, add idle time for unused slots in the cycle.
4. Return total time.

This ensures that the same task is always separated by at least n intervals.

Sabse zyada frequent task ko pehle schedule karo,
kyunki wahi sabse zyada idle create karta hai.

Isliye:

- **Max Heap (highest frequency first)**
- **Cycle = n + 1** (ek task + uske baad n cooldown slots)

Code

```
#include <bits/stdc++.h>

using namespace std;


class Solution {

public:

    int leastInterval(vector<char>& tasks, int n) {

        unordered_map<char,int> freq;

        for(char t : tasks)

            freq[t]++;


        priority_queue<int> maxHeap;

        for(auto it : freq)

            maxHeap.push(it.second);


        int time = 0;
```

```
while(!maxHeap.empty()) {
    vector<int> temp;
    int cycle = n + 1;
    int i = 0;

    while(i < cycle && !maxHeap.empty()) {
        int cnt = maxHeap.top();
        maxHeap.pop();
        cnt--;
        if(cnt > 0)
            temp.push_back(cnt);
        time++;
        i++;
    }

    for(int x : temp)
        maxHeap.push(x);

    if(maxHeap.empty())
        break;

    time += (cycle - i);
}
```

```
        return time;
    }
};


int main() {
    Solution obj;
    vector<char> tasks = {'A','A','A','B','B','B'};
    int n = 2;
    cout << obj.leastInterval(tasks, n);
    return 0;
}
```

Complexity Analysis

- Time Complexity: `O(N log K)`
  `N` is the number of tasks, `K` is the number of unique tasks.

- Space Complexity: `O(K)`
  For frequency map and max heap.

10. Hands of Straights

You are given an array `hand` where each element represents the value of a card. You are also given an integer `groupSize`.
Your task is to check whether it is possible to divide all cards into groups such that:

- Each group has exactly `groupSize` cards
- The values in each group are **consecutive integers**

Example:
Input: `hand = [1,2,3,6,2,3,4,7,8], groupSize = 3`
One valid grouping is `[1,2,3] , [2,3,4] , [6,7,8]`
So the answer is `true`.

Input: `hand = [1,2,3,4,5], groupSize = 4`
It is not possible to divide all cards into groups of 4 consecutive numbers.
So the answer is `false`.

Algorithm

We try to always form groups starting from the **smallest available card**.

Steps:

1. If the total number of cards is not divisible by `groupSize`, return false.
2. Store the frequency of each card in a sorted map.
3. Iterate through the map from the smallest card.
4. For each card with frequency greater than 0:
    - Let this card be the start of a group.
    - Try to form `count` groups of size `groupSize` starting from this card.
    - For each required consecutive card:
        - If it does not exist or its frequency is less than `count`, return false.
        - Otherwise, decrease its frequency by `count`.
5. If all groups are formed successfully, return true.

This greedy approach works because starting from the smallest card avoids breaking the consecutive order.

Hamesha sabse chhote available card se group banana shuru karo.

- Agar tum smallest se start nahi karoge,
- to wo chhota card baad me kisi group me fit hi nahi hoga.

Isi wajah se `map` use hota hai (sorted order).

Code

```
#include <bits/stdc++.h>

using namespace std;


class Solution {

public:

    bool isNStraightHand(vector<int>& hand, int groupSize) {

        if (hand.size() % groupSize != 0)

            return false;


        map<int,int> freq;

        for(int card : hand)

            freq[card]++;


        for(auto it = freq.begin(); it != freq.end(); it++) {
```

```
            if(it->second == 0) continue;

            int start = it->first;
            int count = it->second;

            for(int i = 0; i < groupSize; i++) {
                if(freq[start + i] < count)
                    return false;
                freq[start + i] -= count;
            }
        }
        return true;
    }
};

int main() {
    Solution sol;

    vector<int> hand1 = {1,2,3,6,2,3,4,7,8};
    int groupSize1 = 3;
    cout << sol.isNStraightHand(hand1, groupSize1) << endl;

    vector<int> hand2 = {1,2,3,4,5};
```

```
    int groupSize2 = 4;

    cout << sol.isNStraightHand(hand2, groupSize2) << endl;


    return 0;
}
```

Complexity Analysis

- Time Complexity: `O(N log N)`
  Inserting cards into the map takes `O(N log N)`, and forming groups also involves map access.

- Space Complexity: `O(N)`
  The map stores frequencies of all cards.

11. Design Twitter

You need to design a simplified version of Twitter where users can post tweets, follow or unfollow other users, and view the 10 most recent tweets in their news feed.
The news feed should contain tweets from the user themself and from users they follow, ordered from most recent to least recent.

Example:
Input:
`postTweet(1,2), postTweet(2,6), getNewsFeed(1)`
Output: `[2]`

After `follow(1,2)`
`getNewsFeed(1)` returns `[6,2]`

Approach

Algorithm

The main challenge is efficiently retrieving the **10 most recent tweets** from multiple users.

Steps:

1. Maintain a map that stores tweets for each user.
   Each tweet is stored as `(timestamp, tweetId)` so that recency is tracked.
2. Maintain another map that stores which users a given user follows.
3. Use a global timestamp that increases every time a tweet is posted.
4. For `postTweet`:
    - Add the tweet with the current timestamp to the user's tweet list.
5. For `follow` and `unfollow`:
    - Update the follower's follow set.
6. For `getNewsFeed`:
    - Use a **min-heap of size at most 10**.
    - Push tweets of the user and all followed users into the heap.
    - If heap size exceeds 10, remove the oldest tweet.
    - Finally, extract tweet IDs from the heap in reverse order to get most recent first.
7. Return the result list.

This works because we only keep the top 10 most recent tweets at any time.

Code

```
#include <bits/stdc++.h>

using namespace std;
```

```
class Twitter {

private:

    unordered_map<int, vector<pair<int,int>>> tweets;

    unordered_map<int, unordered_set<int>> following;

    int time;


public:

    Twitter() {

        time = 0;

    }


    void postTweet(int userId, int tweetId) {

        tweets[userId].push_back({time++, tweetId});

    }


    vector<int> getNewsFeed(int userId) {

        priority_queue<pair<int,int>, vector<pair<int,int>>,
greater<>> pq;


        for(auto &t : tweets[userId]) {

            pq.push(t);

            if(pq.size() > 10) pq.pop();

        }
```

```
    for(int followee : following[userId]) {
        for(auto &t : tweets[followee]) {
            pq.push(t);
            if(pq.size() > 10) pq.pop();
        }
    }

    vector<int> res;
    while(!pq.empty()) {
        res.push_back(pq.top().second);
        pq.pop();
    }
    reverse(res.begin(), res.end());
    return res;
}

void follow(int followerId, int followeeId) {
    following[followerId].insert(followeeId);
}

void unfollow(int followerId, int followeeId) {
    following[followerId].erase(followeeId);
```

```
    }
};

int main() {
    Twitter twitter;

    twitter.postTweet(1, 2);
    twitter.postTweet(2, 6);

    vector<int> a = twitter.getNewsFeed(1);
    for(int x : a) cout << x << " ";
    cout << endl;

    twitter.follow(1, 2);

    vector<int> b = twitter.getNewsFeed(1);
    for(int x : b) cout << x << " ";
    cout << endl;

    twitter.unfollow(1, 2);
    twitter.postTweet(1, 7);

    vector<int> c = twitter.getNewsFeed(1);
```

```
    for(int x : c) cout << x << " ";
    cout << endl;

    return 0;
}
```

Complexity Analysis

- Time Complexity:
  - `postTweet`: `O(1)`
  - `follow`: `O(1)`
  - `unfollow`: `O(1)`
  - `getNewsFeed`: `O(T log 10)` ≈ `O(T)`, where T is total tweets checked
- Space Complexity:
  - `O(N)` for storing all tweets
  - `O(U)` for follow relationships
  - Heap uses `O(10)` extra space

12. Connect N Ropes with Minimum Cost

You are given an array `arr[]` where each element represents the length of a rope.
Your task is to connect all ropes into a single rope with the **minimum total cost**.
The cost of connecting two ropes is equal to the **sum of their lengths**.

Example:
Input: `arr = [4, 3, 2, 6]`
One optimal way is:

- Connect 2 and 3 → cost 5, ropes become `[4,5,6]`
- Connect 4 and 5 → cost 9, ropes become `[6,9]`
- Connect 6 and 9 → cost 15
  Total cost = `5 + 9 + 15 = 29`

If there is only one rope, no cost is required.

Approach 1: Naive Approach (Greedy with Repeated Sorting)

Algorithm

The idea is to always connect the two shortest ropes first.

Steps:

1. While more than one rope exists:
2. Sort the array.
3. Take the two smallest ropes.
4. Remove them from the array.
5. Add their sum to total cost.
6. Insert the combined rope back into the array.
7. Continue until only one rope remains.

Code

```cpp
#include <bits/stdc++.h>
using namespace std;

int minCost(vector<int>& arr) {
    int totalCost = 0;

    while (arr.size() > 1) {
        sort(arr.begin(), arr.end());

        int first = arr[0];
        int second = arr[1];

        arr.erase(arr.begin());
        arr.erase(arr.begin());

        int cost = first + second;
        totalCost += cost;

        arr.push_back(cost);
    }
    return totalCost;
}
```

```
int main() {

    vector<int> ropes = {4, 3, 2, 6};

    cout << minCost(ropes);

    return 0;

}
```

Complexity Analysis

- Time Complexity: `O(n^2 log n)`
  Sorting is done repeatedly for each merge.

- Space Complexity: `O(n)`
  Used for storing the ropes.

Approach 2: Optimal Approach (Greedy with Min Heap)

Algorithm

This approach improves efficiency using a **min heap**.

Steps:

1. Insert all rope lengths into a min heap.
2. Initialize total cost as 0.
3. While heap size is greater than 1:
    - Extract the two smallest ropes.
    - Add their sum to total cost.
    - Push the combined rope back into the heap.
4. Return the total cost.

This works because always merging the two smallest ropes minimizes the cost at each step.

Code

```
#include <bits/stdc++.h>

using namespace std;


int minCost(vector<int>& arr) {

    priority_queue<int, vector<int>, greater<int>> pq(arr.begin(),
arr.end());

    int res = 0;


    while (pq.size() > 1) {

        int first = pq.top(); pq.pop();

        int second = pq.top(); pq.pop();


        int cost = first + second;

        res += cost;

        pq.push(cost);

    }

    return res;

}


int main() {

    vector<int> ropes = {4, 3, 2, 6};
```

```
    cout << minCost(ropes);

    return 0;

}
```

Complexity Analysis

- Time Complexity: `O(n log n)`
  Each heap operation takes `log n`, done n times.

- Space Complexity: `O(n)`
  Min heap stores all rope lengths.

13. Kth Largest Element in a Stream of Running Integers

You are given an integer k and an initial stream of integers.
You need to design a class that can continuously return the **kth largest element** after inserting new values into the stream.

The kth largest element is based on **sorted order**, not distinct values.

Example:
Initial stream = `[1, 2, 3, 4]`, `k = 3`
Sorted order = `[1, 2, 3, 4]`
3rd largest = 3

After adding new values, the stream grows and the kth largest must be updated each time.

Approach: Min Heap of Size K

Algorithm

To efficiently track the kth largest element, we maintain a **min heap of size k**.

Key idea:

- The heap always stores the **k largest elements seen so far**.
- The smallest element in this heap is the **kth largest overall**.

Steps:

1. Create a min heap.
2. Insert elements from the initial stream one by one into the heap.
3. If heap size becomes greater than k, remove the smallest element.
4. When a new value is added:
    - Insert it into the heap.
    - If heap size exceeds k, remove the smallest element.
5. The top of the min heap is always the kth largest element.

Code

```
#include <bits/stdc++.h>

using namespace std;


class KthLargest {

    priority_queue<int, vector<int>, greater<int>> minHeap;

    int k;


public:

    KthLargest(int k, vector<int>& nums) {
```

```
        this->k = k;
        for(int num : nums) {
            minHeap.push(num);
            if(minHeap.size() > k)
                minHeap.pop();
        }
    }

    int add(int val) {
        minHeap.push(val);
        if(minHeap.size() > k)
            minHeap.pop();
        return minHeap.top();
    }
};

int main() {
    vector<int> nums = {1, 2, 3, 4};
    KthLargest obj(3, nums);

    cout << obj.add(5) << endl; // 3
    cout << obj.add(2) << endl; // 3
    cout << obj.add(7) << endl; // 4
```

```
    return 0;
}
```

Complexity Analysis

- Time Complexity: `O(log k)` per insertion
  Each add operation involves heap insertion and possible deletion.

- Space Complexity: `O(k)`
  The heap stores only k elements at any time.

14. Maximum Sum Combination

You are given two integer arrays `nums1` and `nums2`, and an integer `k`.
A valid combination is formed by picking **one element from `nums1` and one element from `nums2`** and adding them.
Your task is to return the **top k maximum sum combinations** in **non-increasing order**.

Example:
Input: `nums1 = [7, 3], nums2 = [1, 6], k = 2`
Possible sums: `7+1=8, 7+6=13, 3+1=4, 3+6=9`
Sorted sums: `[13, 9, 8, 4]`
Output: `[13, 9]`

Approach 1: Brute Force Approach

Algorithm

We generate **all possible pair sums** and then select the largest k values.

Steps:

1. Create an empty array to store all sums.
2. Loop through every element in nums1.
3. For each element in nums1, loop through every element in nums2.
4. Add the pair sum to the list.
5. Sort the list in descending order.
6. Return the first k elements.

Code

```
#include <bits/stdc++.h>

using namespace std;


class Solution {

public:

    vector<int> maxCombinations(vector<int>& nums1, vector<int>&
nums2, int k) {

        vector<int> allSums;


        for(int i = 0; i < nums1.size(); i++) {

            for(int j = 0; j < nums2.size(); j++) {

                allSums.push_back(nums1[i] + nums2[j]);

            }

        }
```

```
        sort(allSums.begin(), allSums.end(), greater<int>());

        vector<int> result(allSums.begin(), allSums.begin() + k);
        return result;
    }
};

int main() {
    Solution obj;
    vector<int> nums1 = {7, 3};
    vector<int> nums2 = {1, 6};
    int k = 2;

    vector<int> res = obj.maxCombinations(nums1, nums2, k);
    for(int x : res) cout << x << " ";
    return 0;
}
```

Complexity Analysis

- Time Complexity: $O(n*m \log(n*m))$
  All $n*m$ sums are generated and sorted.
- Space Complexity: $O(n*m)$
  All pair sums are stored.

Approach 2: Optimal Approach (Using Max Heap)

Algorithm

Instead of generating all combinations, we only explore the **largest possible sums first**.

Steps:

1. Sort both arrays in descending order.
2. Use a max heap storing `(sum, i, j)` where `sum = nums1[i] + nums2[j]`.
3. Push the initial combination `(0,0)` into the heap.
4. Maintain a set to avoid revisiting index pairs.
5. Repeat k times:
    - Pop the maximum sum from heap and add it to result.
    - Push `(i+1, j)` and `(i, j+1)` if valid and not visited.
6. Return the result.

This ensures we only process necessary combinations.

Code

```
#include <bits/stdc++.h>

using namespace std;


class Solution {

public:

    vector<int> maxCombinations(vector<int>& nums1, vector<int>&
nums2, int k) {
```

```
sort(nums1.begin(), nums1.end(), greater<int>());
sort(nums2.begin(), nums2.end(), greater<int>());

priority_queue<tuple<int,int,int>> maxHeap;
set<pair<int,int>> visited;

maxHeap.push({nums1[0] + nums2[0], 0, 0});
visited.insert({0,0});

vector<int> result;

while(k-- && !maxHeap.empty()) {
    auto [sum, i, j] = maxHeap.top();
    maxHeap.pop();

    result.push_back(sum);

    if(i + 1 < nums1.size() && !visited.count({i+1, j})) {
        maxHeap.push({nums1[i+1] + nums2[j], i+1, j});
        visited.insert({i+1, j});
    }

    if(j + 1 < nums2.size() && !visited.count({i, j+1})) {
```

```
                maxHeap.push({nums1[i] + nums2[j+1], i, j+1});
                visited.insert({i, j+1});
            }
        }
        return result;
    }
};

int main() {
    Solution sol;
    vector<int> nums1 = {7, 3};
    vector<int> nums2 = {1, 6};
    int k = 2;

    vector<int> res = sol.maxCombinations(nums1, nums2, k);
    for(int x : res) cout << x << " ";
    return 0;
}
```

Complexity Analysis

- Time Complexity: $O(k \log k)$
  Each heap operation costs $\log k$, repeated k times.

- Space Complexity: `O(k)`
  Heap and visited set store at most k elements.

Har iteration:

1. Heap se **current maximum sum** nikalo
2. Uske **neighbors** push karo:
   - `(i+1, j)`
   - `(i, j+1)`

Dry Run

Input

```
nums1 = [7, 3]

nums2 = [6, 1]

k = 2
```

Initial

Heap:

```
(13, 0, 0)  // 7 + 6
```

Result: [ ]

Iteration 1

Pop:

```
(13, 0, 0)
```

Result:

`[13]`

Push neighbors:

- `(1,0)` → `3 + 6 = 9`
- `(0,1)` → `7 + 1 = 8`

Heap:

`(9,1,0), (8,0,1)`

Iteration 2

Pop:

`(9,1,0)`

Result:

`[13, 9]`

k = 0 → stop

Final Answer

`[13, 9]`

15. Find Median from Data Stream

You need to design a data structure that supports adding numbers from a stream and finding the median at any point.
The median is the middle element in sorted order.
If the total number of elements is even, the median is the average of the two middle elements.

Example:
Input: addNum(1), addNum(2), addNum(3)
Sorted: `[1,2,3]` → Median = 2

Input: addNum(1), addNum(6)
Sorted: `[1,6]` → Median = `(1+6)/2 = 3.5`

Approach 1: Brute Force Approach

Algorithm

We store all numbers and sort them whenever the median is requested.

Steps:

1. Maintain a list of all numbers.
2. On `addNum`, insert the number into the list.
3. On `findMedian`:
    - Sort the list.
    - If size is odd, return the middle element.
    - If size is even, return the average of the two middle elements.

Code

```
#include <bits/stdc++.h>
```

```
using namespace std;

class MedianFinder {
    vector<int> nums;

public:
    MedianFinder() {
    }

    void addNum(int num) {
        nums.push_back(num);
    }

    double findMedian() {
        sort(nums.begin(), nums.end());
        int n = nums.size();

        if(n % 2 == 1)
            return nums[n / 2];

        return (nums[n / 2 - 1] + nums[n / 2]) / 2.0;
    }
};
```

```
int main() {

    MedianFinder mf;

    mf.addNum(1);

    mf.addNum(2);

    mf.addNum(3);

    cout << mf.findMedian() << endl;

    return 0;

}
```

Complexity Analysis

- Time Complexity: `O(N log N)` for `findMedian` due to sorting.
- Space Complexity: `O(N)` to store all elements.

Approach 2: Optimal Approach (Two Heaps)

Algorithm

We use two heaps to maintain balance:

- A **max heap** for the smaller half.
- A **min heap** for the larger half.

Rules:

- The size of max heap is either equal to or one greater than min heap.

- All elements in max heap are ≤ elements in min heap.

Steps:

1. Insert the new number into max heap.
2. Move the largest element from max heap to min heap.
3. If min heap has more elements, move its top back to max heap.
4. To find median:
    - If both heaps have equal size, return average of their tops.
    - Otherwise, return the top of max heap.

Code

```
#include <bits/stdc++.h>

using namespace std;


class MedianFinder {

    priority_queue<int> maxHeap;

    priority_queue<int, vector<int>, greater<int>> minHeap;


public:

    MedianFinder() {

    }
```

```
    void addNum(int num) {
        maxHeap.push(num);
        minHeap.push(maxHeap.top());
        maxHeap.pop();

        if(minHeap.size() > maxHeap.size()) {
            maxHeap.push(minHeap.top());
            minHeap.pop();
        }
    }

    double findMedian() {
        if(maxHeap.size() == minHeap.size())
            return (maxHeap.top() + minHeap.top()) / 2.0;

        return maxHeap.top();
    }
};

int main() {
    MedianFinder mf;
    mf.addNum(1);
    mf.addNum(6);
```

```
    cout << mf.findMedian() << endl;

    mf.addNum(3);

    cout << mf.findMedian() << endl;

    return 0;

}
```

Complexity Analysis

- Time Complexity:
    - `addNum`: `O(log N)`
    - `findMedian`: `O(1)`
- Space Complexity: `O(N)` for storing elements in heaps.

Numbers ko do halves me baant do
left half (chhota) → maxHeap
right half (bada) → minHeap

Taaki beech ka element turant mil jaaye.

🔑 Do heaps ka role

1️⃣ maxHeap (left half)

- Isme **chhote elements**
- Top = **left half ka sabse bada**
- Median yahin se aata hai (odd case)

2️⃣ minHeap (right half)

- Isme **bade elements**
- Top = **right half ka sabse chhota**

🔒 Invariants (ye hamesha true rehna chahiye)

1. `maxHeap.size() == minHeap.size()`
   ya
   `maxHeap.size() == minHeap.size() + 1`

2. `maxHeap.top() ≤ minHeap.top()`

Agar ye dono sahi hain → median nikalna trivial.

🔹 addNum(num) ka REAL logic

Code:

```
void addNum(int num) {

    maxHeap.push(num);

    minHeap.push(maxHeap.top());

    maxHeap.pop();


    if(minHeap.size() > maxHeap.size()) {

        maxHeap.push(minHeap.top());

        minHeap.pop();

    }

}
```

Explanation:

Step 1: Pehle maxHeap me daalo

```
maxHeap.push(num);
```

👉 Kyunki initially hum assume karte hain:

naya number left half ka hissa ho sakta hai

Step 2: maxHeap ka largest minHeap me bhejo

```
minHeap.push(maxHeap.top());
```

```
maxHeap.pop();
```

👉 Iska matlab:

- Left half me jo **sabse bada** hai
- Usse right half me shift kar do

Isse ensure hota hai:

```
maxHeap ke saare elements ≤ minHeap ke saare elements
```

Step 3: Size balance karo

```
if(minHeap.size() > maxHeap.size()) {
    maxHeap.push(minHeap.top());
    minHeap.pop();
}
```

👉 Rule:

- maxHeap ko **equal ya ek zyada** element chahiye

Agar minHeap bada ho gaya → ek element wapas bhej do.

🔥 findMedian() ka logic

```
if(size equal)
    median = (maxHeap.top + minHeap.top) / 2
else
    median = maxHeap.top
```

Kyun?

- Odd count → beech ka element left half me hota hai
- Even count → beech ke do elements = dono heaps ke top

🧪 FULL DRY RUN

Start

```
maxHeap = []
minHeap = []
```

---

addNum(1)

Step 1:

`maxHeap = [1]`

Step 2:

`minHeap = [1]`

`maxHeap = []`

Step 3 (rebalance):

`maxHeap = [1]`

`minHeap = []`

Median:

1

---

addNum(6)

Step 1:

`maxHeap = [6,1]`

Step 2:

`minHeap = [6]`

`maxHeap = [1]`

Sizes equal → ok

Median:

`(1 + 6) / 2 = 3.5`

---

addNum(3)

Step 1:

`maxHeap = [3,1]`

Step 2:

`minHeap = [3,6]`

`maxHeap = [1]`

Step 3 (rebalance):

`maxHeap = [3,1]`

`minHeap = [6]`

Median:

3

16. Top K Frequent Elements in an Array

Given an array `arr[]` and an integer k, find the **top k elements with the highest frequency**. If multiple elements have the same frequency, **the larger element should be preferred**.

Example:
Input: `arr = [3,1,4,4,5,2,6,1], k = 2`
Frequencies: `4→2, 1→2, others→1`
Output: `[4, 1]`

Input: `arr = [7,10,11,5,2,5,5,7,11,8,9], k = 4`
Output: `[5, 11, 7, 10]`

Approach 1: Naive Approach (Hash Map + Sorting)

Algorithm

1. Count frequency of each element using a hash map.
2. Store `(element, frequency)` pairs in a vector.
3. Sort the vector:
    - Higher frequency first.
    - If frequency is same, larger element first.
4. Return the first k elements.

Code

```
#include <bits/stdc++.h>
```

```
using namespace std;


//compare(a, b) ka matlab hota hai:
     //"Kya a ko b se pehle aana chahiye?"
   • //true → a pehle aayega
   • //false → b pehle aayega
static bool compare(pair<int,int>& p1, pair<int,int>& p2) {
    if(p1.second == p2.second)
        return p1.first > p2.first;
    return p1.second > p2.second;
}


vector<int> topKFreq(vector<int>& arr, int k) {
    unordered_map<int,int> mp;
    for(int x : arr) mp[x]++;


    vector<pair<int,int>> freq(mp.begin(), mp.end());
    sort(freq.begin(), freq.end(), compare);


    vector<int> res;
    for(int i = 0; i < k; i++)
        res.push_back(freq[i].first);
    return res;
```

```
}


int main() {
    vector<int> arr = {3,1,4,4,5,2,6,1};
    int k = 2;
    vector<int> res = topKFreq(arr, k);
    for(int x : res) cout << x << " ";
    return 0;
}
```

Complexity Analysis

- Time Complexity: `O(n + d log d)`
- Space Complexity: `O(d)`
  (d = number of distinct elements)

Approach 2: Expected Approach 1 (Hash Map + Min Heap)

Algorithm

1. Count frequencies using a hash map.
2. Use a **min heap** of size k storing `(frequency, element)`.
3. Push each map entry into the heap.
4. If heap size exceeds k, remove the smallest frequency element.
5. Extract elements from heap in reverse order.

Code

```
#include <bits/stdc++.h>

using namespace std;


vector<int> topKFreq(vector<int>& arr, int k) {

    unordered_map<int,int> mp;

    for(int x : arr) mp[x]++;


    priority_queue<pair<int,int>, vector<pair<int,int>>,
greater<pair<int,int>>> pq;


    for(auto p : mp) {

        pq.push({p.second, p.first});

        if(pq.size() > k)

            pq.pop();

    }


    vector<int> res(k);

    for(int i = k-1; i >= 0; i--) {

        res[i] = pq.top().second;

        pq.pop();

    }

    return res;
```

```
}


int main() {
    vector<int> arr = {3,1,4,4,5,2,6,1};
    int k = 2;
    vector<int> res = topKFreq(arr, k);
    for(int x : res) cout << x << " ";
    return 0;
}
```

Complexity Analysis

- Time Complexity: `O(n + k log k)`
- Space Complexity: `O(d)`

Approach 3: Expected Approach 2 (Counting Sort / Buckets)

Algorithm

1. Count frequency of each element.
2. Find maximum frequency.
3. Create buckets where index = frequency.
4. Store elements in corresponding buckets.
5. Traverse buckets from high to low frequency.
6. If same frequency, sort elements in descending order.

7. Collect first k elements.

Code

```
#include <bits/stdc++.h>

using namespace std;


vector<int> topKFreq(vector<int>& arr, int k) {

    unordered_map<int,int> freq;

    for(int x : arr) freq[x]++;


    int maxFreq = 0;

    for(auto p : freq)

        maxFreq = max(maxFreq, p.second);


    vector<vector<int>> buckets(maxFreq + 1);

    for(auto p : freq)

        buckets[p.second].push_back(p.first);


    vector<int> res;

    for(int i = maxFreq; i >= 1; i--) {

        sort(buckets[i].begin(), buckets[i].end(), greater<int>());

        for(int x : buckets[i]) {

            res.push_back(x);
```

```
            if(res.size() == k)
                return res;
        }
    }
    return res;
}

int main() {
    vector<int> arr = {3,1,4,4,5,2,6,1};
    int k = 2;
    vector<int> res = topKFreq(arr, k);
    for(int x : res) cout << x << " ";
    return 0;
}
```

Complexity Analysis

- Time Complexity: `O(n log n)`
- Space Complexity: `O(n)`

Approach 4: Alternate Approach (Quick Select)

Algorithm

1. Count frequencies.

2. Store distinct elements.
3. Use QuickSelect to position top k frequent elements at the end.
4. Sort only those k elements:
    - Higher frequency first.
    - If equal frequency, larger value first.
5. Return the result.

Code

```
#include <bits/stdc++.h>

using namespace std;


int partition(int left, int right, int pivotIdx,

              vector<int>& distinct, unordered_map<int,int>& mp) {


    int pivotFreq = mp[distinct[pivotIdx]];

    int pivotVal = distinct[pivotIdx];


    swap(distinct[pivotIdx], distinct[right]);


    int j = left;

    for(int i = left; i < right; i++) {

        if(mp[distinct[i]] < pivotFreq ||

          (mp[distinct[i]] == pivotFreq && distinct[i] < pivotVal)) {
```

```
            swap(distinct[i], distinct[j]);
            j++;
        }
    }
    swap(distinct[j], distinct[right]);
    return j;
}

void quickselect(int left, int right, int k,
                 vector<int>& distinct, unordered_map<int,int>& mp) {
    if(left >= right) return;

    int pivotIdx = left + rand() % (right - left + 1);
    pivotIdx = partition(left, right, pivotIdx, distinct, mp);

    if(pivotIdx == k) return;
    else if(pivotIdx > k)
        quickselect(left, pivotIdx - 1, k, distinct, mp);
    else
        quickselect(pivotIdx + 1, right, k, distinct, mp);
}

vector<int> topKFreq(vector<int>& arr, int k) {
```

```
    unordered_map<int,int> mp;
    for(int x : arr) mp[x]++;

    vector<int> distinct;
    for(auto p : mp) distinct.push_back(p.first);

    int n = distinct.size();
    quickselect(0, n-1, n-k, distinct, mp);

    sort(distinct.begin()+n-k, distinct.end(), [&](int a, int b){
        if(mp[a] != mp[b]) return mp[a] > mp[b];
        return a > b;
    });

    vector<int> res;
    for(int i = n-k; i < n; i++)
        res.push_back(distinct[i]);
    return res;
}

int main() {
    vector<int> arr = {3,1,4,4,5,2,6,1};
    int k = 2;
```

```
    vector<int> res = topKFreq(arr, k);
    for(int x : res) cout << x << " ";
    return 0;
}
```

Complexity Analysis

- Time Complexity: Average $O(n)$ (QuickSelect), worst $O(n^2)$
- Space Complexity: $O(n)$